Informe del desarrollo de la red propuesta en el desafío #2

Se requiere un programa utilizando POO como medida de solución, que permita a un conjunto de usuarios, iniciar sesión, administrar su perfil, establecer amistades, intercambiar mensajes, y consultar álbumes presentes de la página donde su interacción será totalmente por consola. Dichos procesos tenían como requisitos mínimos:

- No usar contenedores de la STL como por ejemplo vector, map.. etc. Por lo cual se utilizo una plantilla propia Lista<T>, como medida de almacenamiento dicha lista se comporta como un arreglo dinámico que al llegar al limite de capacidad se duplicaba de una manera óptima.
- El programa debía tener memoria permanente, no se puede perder la información al momento de cerrar la ejecución, por lo cual se opto por el uso de archivos de texto ".txt" separados con la información de relevancia o importancia, dicha información fue separada con el símbolo "|" para una mejor separación. Lo cual fue de gran ayuda debido a que dentro de la información personal de los usuarios podría haber símbolos comunes o usuales que dificultarían al programa una detección optima.
- El programa permite ver la información propia del usuario y de los usuarios que fueron catalogados como amigos, por lo cual si quiere ver la información de otros usuarios, solo se observara la que este catalogada como publica
- El programa debe conservar cierta lógica interna, no mandar solicitud de amistad a quien ya es amigo, no solicitudes repetidas, solo se puede mandar mensajes a los amigos, etc.

Se utiliza programación orientada a objetos como base de la solución. Por diseño, se definen las clases Usuario, Mensaje, SolicitudAmistad, Album y Foto, que conservan la información más relevante del programa; esta se encapsula y se valida, en parte dentro de las propias clases y en parte en las funciones del menú, para evitar errores en ejecución. La información se almacena en listas dinámicas mediante la plantilla Lista<T>, que se redimensiona cuando necesita más capacidad y permite acceder a cada elemento por su índice, guardando los objetos por valor. Esto evita tener que crear y liberar cada elemento por separado, como ocurriría con un arreglo de punteros. Las relaciones entre los datos, como las amistades o la pertenencia de un álbum a un usuario, se establecen por id mediante listas paralelas. La clase Datos se encarga de los archivos de texto, y Funciones.h agrupa las operaciones del programa.

Limitaciones del modelo para tener en cuenta. La búsqueda de información es lineal, por ende, entre mayor sea el arreglo de información/datos el programa se vera afectado en su eficacia, además si se llegara a cambiar el sistema de ID de los usuarios habría problemas con la identificación entre uno y otro debido a que se procesaron en la medida que serían únicos.

Diagrama de clases: https://lucid.app/lucidchart/c94bf898-714d-4ca9-9bdf-a150f767f70e/edit?viewport_loc=-1426%2C-583%2C2398%2C1356%2C0_0&invitationId=inv_b22dd5de-f404-4df3-8f39-b568c7aff9dc

Toda la relación se puede sintetizar de la siguiente manera: se genera un álbum, este para crearse debe tener una o más fotos (composición). Mientras que las solicitudes y mensajes están relacionados a los usuarios gracias a un ID(relación) y todas estas clases se unen para conformarse en una lista de información(dependencia) y dichas listas depende de los datos y las funciones para generar el programa(dependencia).

Algoritmos esenciales

GenerarArchivos: Esta función genera los datos que utiliza el programa, como usuarios, mensajes, amistades y solicitudes aleatorias. El proceso se realiza de forma independiente de la lectura y carga de archivos, lo que ayuda a evitar inconvenientes al recuperar la información. Los datos se crean a partir de listas predefinidas de nombres, ciudades y países. También se establece de antemano el contenido de los mensajes por defecto.

Campo: Los datos se almacenan en archivos de texto plano y sus valores están separados por el carácter «|». En las primeras versiones, la extracción se realizaba manualmente, ya que había poca información que procesar: uno o dos ejemplos por caso. Al tener como requisito un mínimo de 1.000 usuarios y para evitar repetir el código de manejo de archivos, se creó la función campo. Esta recibe una línea de texto y el número del campo o columna que se desea consultar; luego, utiliza el carácter «|» como separador para localizar y devolver el dato solicitado en formato string.

Funciones de carga: ya la información existe en la memoria permanente, pero no en el programa, por lo cual usando (campo),Donde atravez de la ruta donde se encuentran los datos, se van recorriendo los correspondientes archivos de textos plano y generando sus respectivos objetos (usuario, mensaje, amistades...) y ya con los objetos creados se añaden a sus respectivas listas.

Funciones de guardado: Estas funciones utilizan las rutas de los archivos y las listas actualizadas para sobrescribir los archivos de texto plano, lo que permite conservar los datos y reflejar los cambios realizados. Cuando el usuario selecciona la opción de cerrar el programa, los archivos TXT se reescriben automáticamente con la información vigente en las listas. Se mantiene el mismo formato de separación entre campos para evitar problemas al volver a ejecutar el programa.

Problemas durante el desarrollo

A lo largo del desarrollo surgieron muchos problemas que se repitieron. Uno de esos aspectos fue la gestión de errores al leer datos por consola. Pese a que el software estaba concebido para ser intuitivo, si el usuario introducía un tipo de dato diferente al requerido, la ejecución se detenía bruscamente y no aceptaba la entrada. Esto podría causar que los cambios efectuados en la sesión se pierdan. Se observó, además, que las fechas no estaban correctamente validadas. El programa esperaba una secuencia de números con el formato «AAAAMMDD», pero admitía valores que no eran posibles, como «99991212», a pesar de que

no correspondieran a una fecha válida. Las funciones darLike y quitarLike de la clase Foto tenían otro inconveniente. Aunque los cambios se mostraban adecuadamente en la interfaz y durante la ejecución, no se mantenían al recargar la información. Finalmente, quedaba la validación de las solicitudes de amistad. Como estas eran administradas a través de identificadores, el programa no verificaba si el usuario receptor ya era amigo del remitente de la solicitud, lo cual facilitaba la creación de solicitudes innecesarias.

Evolución de la solución y consideraciones para futuras modificaciones

La primera clase creada fue Foto, basada en el diagrama de clases e implementada con métodos esenciales como darLike, quitarLike y mostrarFoto. Después de verificar su funcionamiento con objetos individuales, se creó la plantilla dinámica Lista<T>, necesaria para que la clase Album pudiera almacenar y organizar sus fotografías. A partir de ella se desarrolló Album, que incluye métodos para administrar las fotos que lo componen, como el carrusel. Según lo pedido en el enunciado, este permite consultar la información del álbum, conocer su nivel de privacidad (público o amigos) y saber cuántas fotos contiene.

Posteriormente se incorporaron las clases Mensaje y SolicitudAmistad. Como no tenían una relación directa entre sí, se probaron de forma independiente mediante la creación de objetos de cada tipo. Con estas clases disponibles, se desarrolló una versión básica de Usuario, que incluía datos como el identificador, el nombre y el correo electrónico.

Para evitar ingresar manualmente los datos de cada objeto, se creó la clase Datos, encargada de generar información semialeatoria. Estos datos pueden variar cada vez que se generan, pero se conservan después de la primera ejecución, porque la función generadora no vuelve a invocarse si los archivos ya existen. La información guardada en los archivos se carga en memoria usando Lista<T>, tanto para los usuarios como para las amistades, los mensajes y los álbumes.

Por último, se desarrollaron las funciones que integran estas clases y permiten ejecutar los procesos internos del programa. Ya con el sistema funcionando, se hicieron ajustes: se agregó la validación de fechas y el control de solicitudes duplicadas, y se corrigió el guardado de datos para que se conservaran el estado de las solicitudes, los mensajes leídos y los likes de las fotos.

Consideraciones durante la ejecución/modificación

- Si quiere cambiar la información generada por la clase random, se debe tener en cuenta siempre como medio de separación el símbolo “|” para evitar problemas de ejecución o lecturas erróneas de la información permanente
- Es posible que algunas entradas numéricas no cuenten con todas las validaciones correspondientes, por lo cual es posible que el programa se cierre si recibe un parámetro que no halla tenido un manejo de excepción.
- Si posteriormente se quiere incluir una función para eliminar usuarios, se debe tener en cuenta que este solo existe en su correspondiente archivo de texto, por lo cual la información referente a este aun seguirá existiendo en los demás archivos.

- Falla en la lógica de los likes, aunque estos puedan aumentar o disminuir si el usuario ingresa o quita, no se valido una cantidad fija de forma inicial. Por lo cual cualquier usuario puede llevar la cantidad de likes a 0.